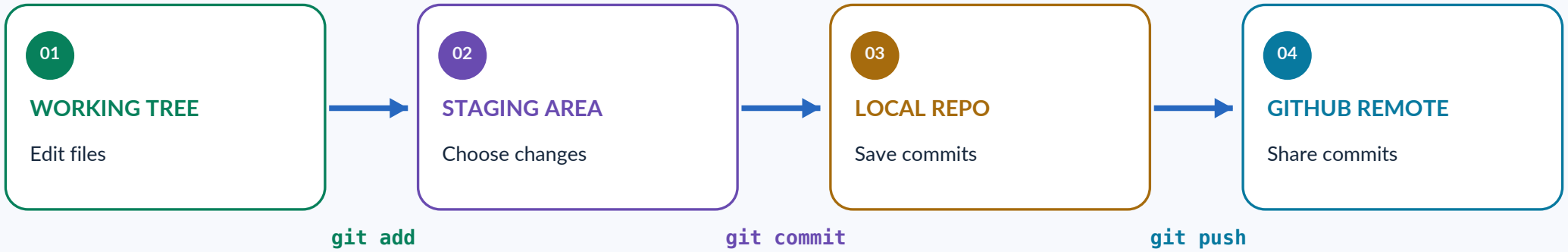
GIT & GITHUB VISUAL WORKFLOW

THE 4-STAGE DAILY GIT WORKFLOW



`Unstage: git restore --staged <file>`

Fetch: download remote history | Pull: fetch + integrate

BASICS

```
git init  - start repository
```
git status - review changes

git log --oneline - see commits

BRANCHING

```
git switch -c feature/x
```
git switch main

git merge feature/x

CONFLICTS

```
Same lines edited on 2 branches
```
Resolve file conflict markers

git add file && git commit

MERGE vs REBASE

```
merge  - preserves existing history
```
rebase - replays commits

Avoid rebasing shared branches

TEMPORARY WORK

```
git stash -u  - save all edits
```
git stash pop - restore + drop

git stash apply - restore + keep

UNDO / RECOVERY

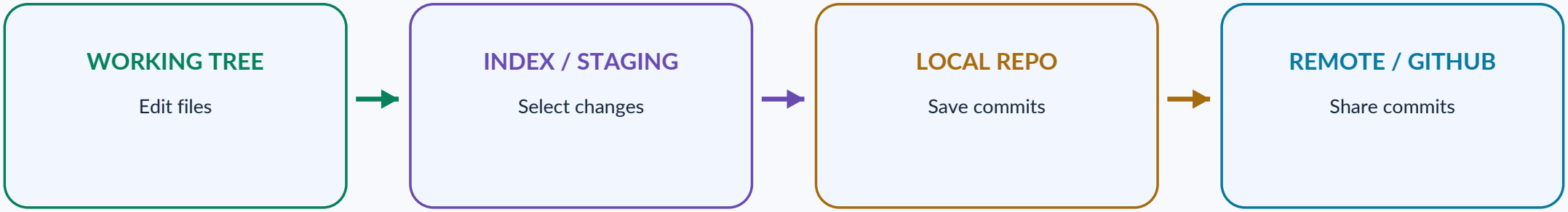
```
git revert HEAD  - safe shared undo
```
git reset --soft HEAD~1

git reflog - find lost commits

MEMORY HOOK `edit -> add -> commit -> push | branch -> PR -> review -> merge`

01 THE GIT MENTAL MODEL

Understand the 4 locations before memorizing commands.



DAILY COMMANDS

In a fresh repository or your existing project

`git init`	Start a new local repo
`git clone <url>`	Copy a remote repo
`git status -sb`	Compact branch + changes
`git add <file>`	Stage one file
`git add .`	Stage changes below current path
`git commit -m "msg"`	Save staged snapshot
`git log --oneline --graph --all`	View a history graph
`git show HEAD`	Inspect last commit
`git diff`	Unstaged changes
`git diff --staged`	Staged changes

WHAT MOVES WHERE?

A command acts on a specific area

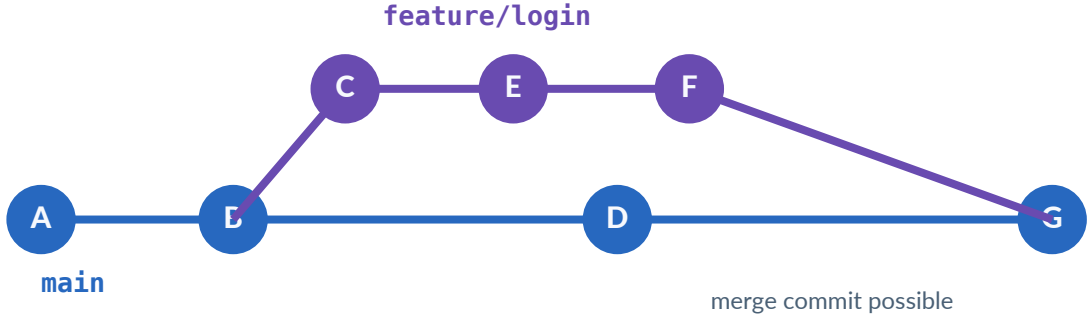
`git add`	Working tree -> index
`git restore --staged x`	Index -> unstaged (keeps file edits)
`git restore x`	Discard unstaged file edits
`git commit`	Index -> new local commit
`git fetch`	Remote -> local tracking refs
`git pull`	Fetch + integrate into current branch
`git push`	Local commits -> remote branch

HEAD = what you currently have checked out.

02 BRANCHES, MERGES & CONFLICTS

Feature development without breaking main.

FEATURE BRANCH VISUAL



Fast-forward: if main did not diverge, Git can just move its pointer forward (no merge commit).

MERGE CONFLICT

Two branches change the same lines

```
<<<<<<< HEAD
name = "main"
=======
name = "feature"
>>>>>>> feature/login
```

1 Open the conflicted file
2 Choose/correct the final content
3 `git add file && git commit`

BRANCHING COMMANDS

`git branch`	List branches
`git switch -c feature/login`	Create + switch to new branch
`git switch main`	Switch to main
`git merge feature/login`	Merge feature into current branch
`git merge --abort`	Cancel in-progress merge
`git branch -d feature/login`	Delete already-merged local branch
`git branch -D feature/login`	Force-delete branch (careful!)

MERGE VS REBASE

MERGE

Preserves existing commits. Can add a merge commit. Best for combining shared history.

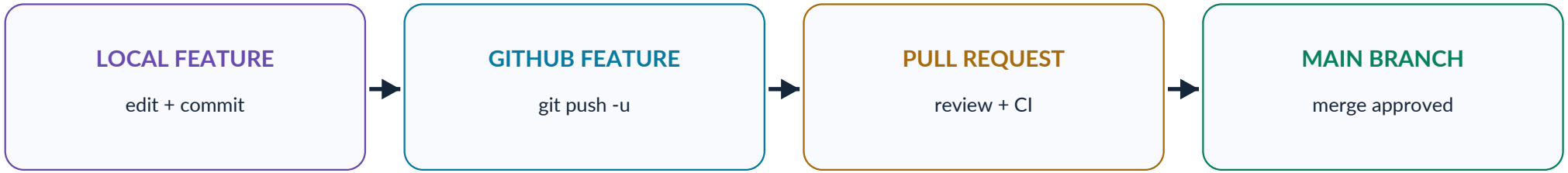
REBASE

Replays commits on a new base and changes their IDs. Keep shared/published history intact.

03 GITHUB & TEAM COLLABORATION

Origin, upstream tracking, fetch/pull, and the pull request loop.

HOW A TEAM SHIPS A CHANGE




```
After merging the PR:  git switch main  &&  git pull --ff-only  &&  git branch -d feature/login
```

GITHUB / REMOTES

`git remote -v`	Show remote names + URLs
`git remote add origin <url>`	Add a remote
`git push -u origin main`	Push + set upstream tracking
`git fetch origin`	Download remote history
`git pull --ff-only`	Only pull if fast-forward possible
`git push`	Push branch with upstream set
`git branch -vv`	See remote tracking info
`git fetch --prune`	Prune stale remote-tracking refs

INTERVIEW DISTINCTIONS

fetch vs pull

fetch only updates remote-tracking refs. pull fetches AND integrates into the checked-out branch.

origin vs upstream

origin is a conventional remote name; upstream may mean tracking branch or a fork's parent remote.

push -u

--set-upstream records default remote branch for future push/pull.

PR vs merge

Pull request = GitHub review process; merge = Git operation combining histories.

```
git pull = fetch + merge/rebase (per configuration)
```

04 UNDO, STASH & RECOVERY

Pick the command by the kind of mistake - not by muscle memory.

WHAT ARE YOU TRYING TO UNDO?

UNSTAGED EDIT

```
git restore file
```

STAGED FILE

```
git restore --staged file
```

LAST LOCAL COMMIT

```
git reset --soft HEAD~1
```

PUSHED BAD COMMIT

```
git revert HEAD
```

WARNING: git reset --hard can destroy uncommitted tracked changes. Check git status first.

RESET VS REVERT

`git reset --soft HEAD~1`
Moves branch back; changes remain staged.

`git reset HEAD~1`
Moves branch back; changes unstaged (mixed default).

`git reset --hard HEAD~1`
Moves branch back; tracked changes discarded.

`git revert <hash>`
New commit reverses selected commit.

`git commit --amend`
Edit last commit (avoid if shared).

STASH + REFLOG RESCUE

`git stash -u`	Save tracked + untracked edits
`git stash list`	List saved work
`git stash pop`	Apply; drop stash on success
`git stash apply`	Apply; keep stash

Lost a local commit after reset?

```
git reflog
git cherry-pick <recovered-hash>
```

Reflog is local and expires eventually; recovery is not guaranteed after pruning.

05 ADVANCED HISTORY OPERATIONS

Clean commits, locate regressions, and port precise fixes.

INTERACTIVE REBASE

Rewrite only your unpublished/local commits

```
git rebase -i HEAD~3
```

```
pick a1b2c3    add endpoint
squash d4e5f6  fix tests
reword 123abc  better commit message
drop 789fed    accidental debug print
```

pick = keep | squash = combine | fixup = combine, discard message | reword = rename | drop = remove

```
git rebase --continue / --abort
```

GIT BISECT = BINARY SEARCH

Find the commit that introduced a bug




```
git bisect start

git bisect bad

git bisect good <good-hash>
```

test checkout; mark good or bad

```
git bisect reset
```

CHERRY-PICK / AMEND

```
git cherry-pick <hash>
```

Apply one specific commit onto the current branch (new commit ID). Useful for production hotfix backports.

```
git commit --amend
```

Edit the last local commit; it gets a new ID.

INSPECT & INVESTIGATE

`git blame file`	Who last changed each line?
`git log -- file`	History of one file
`git show <hash>`	View one commit / patch
`git grep "needle"`	Search tracked content

06 ADVANCED DEVOPS TOOLKIT

Parallel work, large repos, automation and releases.

WORKTREES

`git worktree add ../hotfix -b hotfix`
New branch in another working directory

`git worktree list`
List linked worktrees

`git worktree remove ../hotfix`
Remove finished linked worktree

Use: investigate prod bug while main work remains open.

HOOKS

`.git/hooks/pre-commit`
Local checks before commit

`git config core.hooksPath .githooks`
Use versioned hooks directory

`CI on GitHub`
Enforce checks server-side too

Hooks are local; do NOT treat them as reliable security boundaries.

TAGS + RELEASES

`git tag -a v1.2.0 -m "Release"`
Create annotated release tag

`git push origin v1.2.0`
Publish tag

`git tag -l`
List local tags

Tags mark specific commits; branches move with new commits.

SPARSE CHECKOUT

`git sparse-checkout init --cone`
Enable sparse working tree

`git sparse-checkout set infra apps`
Limit checked-out directories

`git sparse-checkout disable`
Return to full working tree

Use: massive monorepos. This does not automatically reduce full history.

SUBMODULES

`git submodule add <url> vendor/lib`
Track a repo as a nested dependency

`git submodule update --init --recursive`
Initialize nested repos

`git submodule status`
Show pinned commits

Submodules pin a separate repository at a specific commit.

GIT LFS

`git lfs install`
Enable LFS hooks

`git lfs track "*.bin"`
Track large binary objects

`git add .gitattributes`
Commit pattern rules

Use for large assets; needs Git LFS installed and host support.

07 SECURITY, INSPECTION & SAFETY

The deeper topics that separate tool users from confident engineers.

GIT INTERNALS IN 45 SECONDS

`blob`	Content of a file
`tree`	Folder listing / snapshot structure
`commit`	Tree + parents + metadata
`HEAD`	Checked-out branch pointer or commit
`origin/main`	Local remote-tracking ref

SAFE HISTORY REWRITES

`git push --force-with-lease`

Safer than --force: reject overwrite if the tracked remote tip changed unexpectedly. Still coordinate with your team.

`git branch backup/before-rewrite`

Create a safety branch before risky local history edits.

SECURE GIT HYGIENE

Secrets

Never commit tokens, keys or .env secrets. If committed, revoke/rotate; deleting a file later does not remove history.

.gitignore

Prevents UNTRACKED matching files from being added; does not untrack already tracked files.

Tracked removal

git rm --cached <file> (then commit ignore rule)

SSH / signing

Use SSH keys or approved credentials; sign commits if required by team policy.

TACTICAL COMMANDS

`git add -p`	Stage selected hunks (patch mode)
`git diff main...feature`	Review changes since merge base
`git switch --detach <hash>`	Inspect an old commit (detached HEAD)
`git clean -nd`	Preview removal of untracked files
`git clean -fd`	Actually remove untracked files - destructive
`git fetch --prune`	Remove stale remote-tracking refs

08 INTERVIEW CHEAT SHEET

The question -> the right Git action. Scan in 90 seconds.

REAL-WORLD SCENARIOS: NAME THE TOOL

Need a fresh local repository?

```
git init
```

Stage only a few lines of a file?

```
git add -p
```

Switch and create a feature branch?

```
git switch -c feature/x
```

Need teammate commits, no integration?

```
git fetch origin
```

Undo a staged file, keep edits?

```
git restore --staged file
```

Pause unfinished tracked + untracked work?

```
git stash -u
```

Bad commit already pushed to main?

```
git revert <hash>
```

Restore commit lost after local reset?

```
git reflog; git cherry-pick <hash>
```

Move one hotfix across branches?

```
git cherry-pick <hash>
```

Combine 3 unpublished commits?

```
git rebase -i HEAD~3
```

Find exactly which commit broke tests?

```
git bisect
```

Work on 2 branches simultaneously?

```
git worktree add ...
```

Check who changed a specific line?

```
git blame file
```

Push rewritten private branch safely?

```
git push --force-with-lease
```

Prevent secrets from reaching history?

```
Ignore + scan; rotate if exposed
```

Merge conflict after rebase?

```
Fix; git add; git rebase --continue
```

MEMORY HOOK: add = stage | commit = local | push = remote | revert = safe undo | reflog = rescue

Official references: git-scm.com/docs | Pro Git: git-scm.com/book